

For assignment 1, the first tasks are to:

- complete the [queue simulation](#), and make sure that results for $d=1$ correspond to theory;
 - Hint: before running the bash scripts, run `queue_sim.py` on its own. There are plenty of parameters you can play with, see them by running `python queue_sim.py -h`.
- reproduce with experimental data the plot in the slides ("Theoretical Queue Length" for Supermarket load balancing):
 - you'll need to check *at regular intervals* (not, for example, at every job arrival/completion) the queue size (for inspiration look at `MonitorSIR` from `sir.py`);
 - note that on the y-axis, you have the **fraction of queues with size $\geq x$** (if you don't know how to do this, a hint is that you're doing something similar to a prefix sum--search for this concept);
 - verify to what extent your results in terms of queue lengths correspond to those shown in the lesson slides (remember that those are theoretical asymptotic results for n and $\max-t$ tending to infinity);
 - keeping in memory or writing to disk the length of each queue will be very slow; use a suitable data structure that maps lengths to number of cases observed, such as, for example, [collections.Counter](#);
 - remember that you can plot the theoretical values yourselves (the equation is in the previous slide).
- Discard the memoryless assumption and use a [Weibull distribution](#) for job sizes:
 - the Weibull distribution has a `shape` parameter (k in the Wikipedia page) that controls its behavior:
 - with `shape = 1`, we have the same exponential distribution of before, hence memoryless behavior;
 - with `shape < 1` we have a "heavy-tailed" distribution, with few very large jobs and many very small ones (for job size), or periods where many jobs are submitted close to each other (for job inter-arrival time);
 - with `shape > 1` we get towards a "bell-shaped" distribution where values are more likely to be close to the mean;
 - note that we can use the Weibull distribution to model both job inter-arrival time and job size. Use two different parameters to model the shape of each, and reflect on what that means in terms of your workload.
 - use the implementation in `workloads.py` and take as guidance the comments in `queue_sim.py`;
 - verify that the results with `shape = 1` are the same as before.

Once you are done with this, **study the simulator code**, try to find places where things might change, and consider modifying the simulator to answer some kind of question (e.g., "Does the system become more efficient if we make this modification in its behavior?" or "How does the system react to this change in its workload?").

Typical questions you can think about can be of these types:

- how does the system react to a specific situation?
- does this change make the system more efficient? what are the tradeoffs?
- how does the behavior change when we introduce this more realistic variant?

Avoid duplicating source code. For your modification it is generally better to introduce a command-line option you to enable your code modification.

Try your best to find an inspiration (if you really study the code it shouldn't be too hard), but even if you found none, contact **Matteo Dell'Amico** (matteo.dellamico@unige.it) to agree on your modification (remember that emails are not only for 1-to-1 messages: if you have one, put your teammate in CC and use reply-all to include everybody in the mail exchange). Then, work on implementing and evaluating it; consult the documents on the peer review process and guidance for more information on how to proceed.

Last modified: Tuesday, 12 May 2026, 5:14 PM

